

**Momento de Retroalimentación: Módulo 2 Uso de framework o biblioteca de aprendizaje máquina para la implementación de una solución.
(Portafolio Implementación)**

Mauricio Guerrero González A01751436

Grupo 601

Jorge Adolfo Ramírez Uresti

Instituto Tecnológico de Estudios Superiores de Monterrey

09 de septiembre del 2026

1. Dataset empleado

El algoritmo se entrena y evalúa utilizando el dataset Titanic - Machine Learning from Disaster de Kaggle. Se utiliza específicamente el archivo train.csv, que contiene 891 registros de pasajeros y la variable objetivo Survived. Esta variable indica si el pasajero sobrevivió al desastre, donde 0 representa que no sobrevivió y 1 que sobrevivió.

Característica	Valor
Registros totales	891
Variables disponibles	12
Clase 0 - No sobrevivió	549
Clase 1 - Sobrevivió	342
Datos para entrenamiento	712 (80 %)
Datos para prueba	179 (20 %)

Tabla 1: Características principales del dataset utilizado para la implementación.

Tal como se señala en la tabla anterior, el archivo se divide de forma estratificada en 80 % para entrenamiento y 20 % para prueba. Se utiliza una semilla fija de 42 para que la partición sea reproducible.

2. Preparación de los datos

El modelo utiliza las variables Pclass, Age, SibSp, Parch, Fare, Sex y Embarked. Se dejan fuera variables de identificación como PassengerId, Name, Ticket y Cabin, ya que no forman parte de las variables seleccionadas para configurar este clasificador.

Los valores faltantes de las variables numéricas se reemplazan mediante la mediana calculada a partir del conjunto de entrenamiento. Para las variables categóricas se utiliza la categoría más frecuente. Posteriormente, Sex y Embarked se convierten a variables numéricas mediante OneHotEncoder.

3. Configuración del framework y algoritmo

La implementación utiliza scikit-learn, una biblioteca de aprendizaje máquina de Python. El clasificador elegido es DecisionTreeClassifier. Esta clase permite configurar directamente los criterios de división y las restricciones del árbol, demostrando el uso del framework para entrenar, predecir y evaluar un modelo de clasificación.

Parámetro	Configuración	Propósito
criterion	gini	Mide la impureza de los nodos para seleccionar divisiones.
max_depth	4	Limita la profundidad máxima del árbol.
min_samples_split	30	Evita dividir nodos con pocas observaciones.
min_samples_leaf	15	Exige un mínimo de observaciones en cada hoja.
random_state	42	Mantiene reproducible el resultado.

Tabla 2: Parámetros principales utilizados para configurar el árbol de decisión.

La profundidad máxima y los mínimos de división y de hoja se establecen para obtener un árbol controlado y reducir el riesgo de sobreajuste. Cabe destacar que la implementación no construye manualmente el árbol, ya que la estructura, las divisiones y las hojas son generadas por DecisionTreeClassifier.

4. Ejecución y predicciones

Después del entrenamiento, el modelo genera predicciones sobre las 179 observaciones del conjunto de prueba. A continuación se muestran los primeros 20 valores predichos y sus correspondientes clases reales.

Observación	1	2	3	4	5	6	7	8	9	10
Real	0	0	1	0	1	1	1	0	0	0
Predicción	0	0	0	0	1	0	1	0	0	0
Observación	11	12	13	14	15	16	17	18	19	20
Real	0	0	1	0	0	0	0	0	0	1
Predicción	0	0	0	0	0	0	0	0	0	1

Tabla 3: Primeras 20 predicciones generadas por el árbol de decisión y sus valores reales.

5. Resultados de evaluación

La evaluación se realiza sobre el conjunto de prueba mediante la matriz de confusión y cuatro métricas de clasificación: Accuracy, Precision, Recall y F1-score. Estas métricas permiten observar tanto el desempeño global como la capacidad de identificar correctamente la clase de supervivencia.

Métrica	Entrenamiento	Prueba
Accuracy	0.8315	0.7765
Precision	0.8883	0.8085
Recall	0.6410	0.5507
F1-score	0.7447	0.6552

Tabla 4: Métricas obtenidas para evaluar el desempeño del modelo en entrenamiento y prueba.

El Accuracy de 0.7765 indica que el árbol clasifica correctamente 77.65 % de las observaciones del conjunto de prueba. En comparación con el resultado de entrenamiento de 0.8315, existe una reducción de 5.50 puntos porcentuales al evaluar datos no utilizados durante el ajuste, pero el modelo conserva un desempeño útil para la tarea.

Por otro lado, la métrica Precision de 0.8085 indica que, de las observaciones que el modelo clasifica como sobrevivientes, 80.85 % pertenece realmente a esa clase. En este sentido, el modelo genera relativamente pocos falsos positivos, solo 9 casos dentro del conjunto de prueba.

El Recall de 0.5507 es menor que el resto de las métricas y muestra una limitación importante. El modelo identifica correctamente 55.07 % de los pasajeros que realmente sobrevivieron, lo cual

representa los 31 falsos negativos observados en la matriz de confusión.

Finalmente, el F1-score de 0.6552 resume el equilibrio entre Precision y Recall. En esta ocasión su valor muestra que existe una diferencia considerable entre la capacidad del modelo para evitar falsos positivos y su capacidad para recuperar todos los casos de supervivencia.

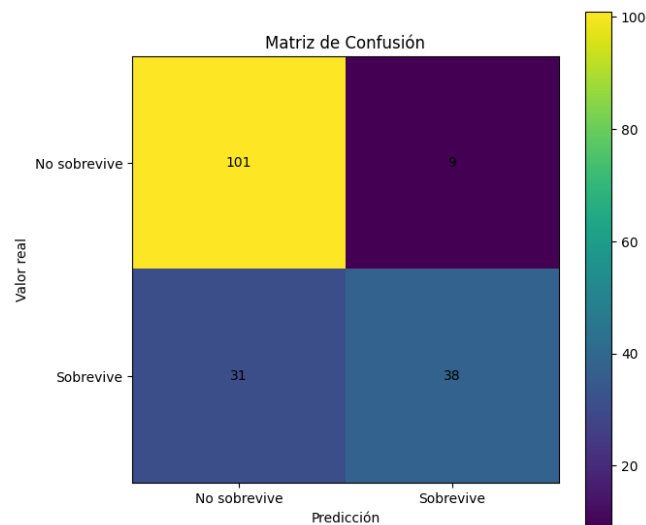


Figura 1: Matriz de confusión obtenida sobre el conjunto de prueba.

La matriz de confusión obtenida es $[[101, 9], [31, 38]]$, considerando las filas como valores reales y las columnas como predicciones. Esto corresponde a 101 verdaderos negativos, 9 falsos positivos, 31 falsos negativos y 38 verdaderos positivos.

El modelo clasifica correctamente 139 de las 179 observaciones de prueba. Los 9 falsos positivos indican casos en los que el modelo predice que el pasajero sobrevivió cuando en realidad no sobrevivió. Los 31 falsos negativos representan pasajeros que sí sobrevivieron, pero que fueron clasificados como no sobrevivientes.

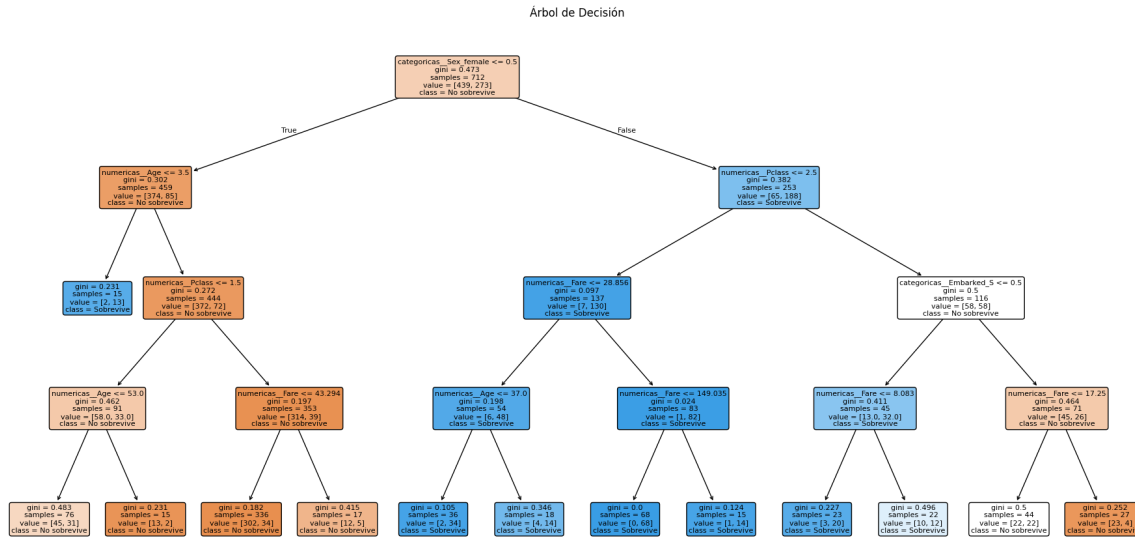


Figura 2: Estructura del árbol de decisión generado por scikit-learn.

La figura anterior muestra la estructura generada por `DecisionTreeClassifier` después del entrenamiento. Las variables categóricas aparecen codificadas como variables binarias y cada nodo contiene la condición utilizada para dividir las observaciones.

6. Conclusión

El árbol de decisión implementado con scikit-learn cumple con el objetivo de clasificar la supervivencia de los pasajeros del Titanic y demuestra el uso de un framework de aprendizaje máquina para configurar, entrenar y ejecutar un algoritmo de clasificación. Además, cabe destacar que la implementación puede ejecutarse de forma independiente desde un archivo .py y no depende de un notebook o de un IDE específico.

Los resultados muestran un desempeño aceptable para una primera configuración: se obtiene un Accuracy de 77.65 % y una Precision de 80.85 % en el conjunto de prueba. La matriz de confusión confirma que la mayoría de las observaciones se clasifican correctamente. Sin embargo, el Recall de 55.07 % muestra que aún existe una cantidad importante de falsos negativos, por lo que el modelo podría mejorar mediante ciertos ajustes de hiperparámetros, selección de variables o con el uso de otras configuraciones del árbol.